GAME DEVELOPMENT PORTFOLIO

포켓몬스터 팬게임

C++와 Win32 API 기반 2D 게임

타일 맵 에디터와 전투 화면 구현

개발 박우영



프로젝트 개요와 담당 역할

C++로 윈도우 기반 게임의 기본 구조와 게임 로직을 구현한 2인 프로젝트입니다.

개발 박우영

게임 엔진과 게임 로직을 구현했습니다. Win32 API와 GDI+를 활용한 그래픽 처리, STL을 활용한 데이터 관리와 알고리즘 구현을 담당했습니다.

기획 윤상혁

스토리와 게임 콘셉트, 플레이 방식, 맵 디자인을 담당했습니다.

주요 구현

타일 맵 에디터, 더블 버퍼링, 객체와 상태 관리, 버튼 콜백, 전투 UI와 HP 표시

개발 기술과 사용 목적

C++와 Win32 API

윈도우 생성, 메시지 처리, 입력 처리와 게임의 기본 실행 구조를 구현했습니다.

GDI+

PNG 리소스를 불러와 비트맵으로 변환하고 게임 화면에 출력했습니다.

STL

vector와 map을 이용해 게임 데이터와 객체를 관리했습니다.

Photoshop과 GitHub

게임 아트의 제작·편집에 Photoshop을 사용하고, GitHub로 버전을 관리했습니다.

포켓몬스터 스프라이트는 The Spriters Resource에서 받은 리소스를 사용했습니다.

스토리 기획과 구현 범위

스토리 기획

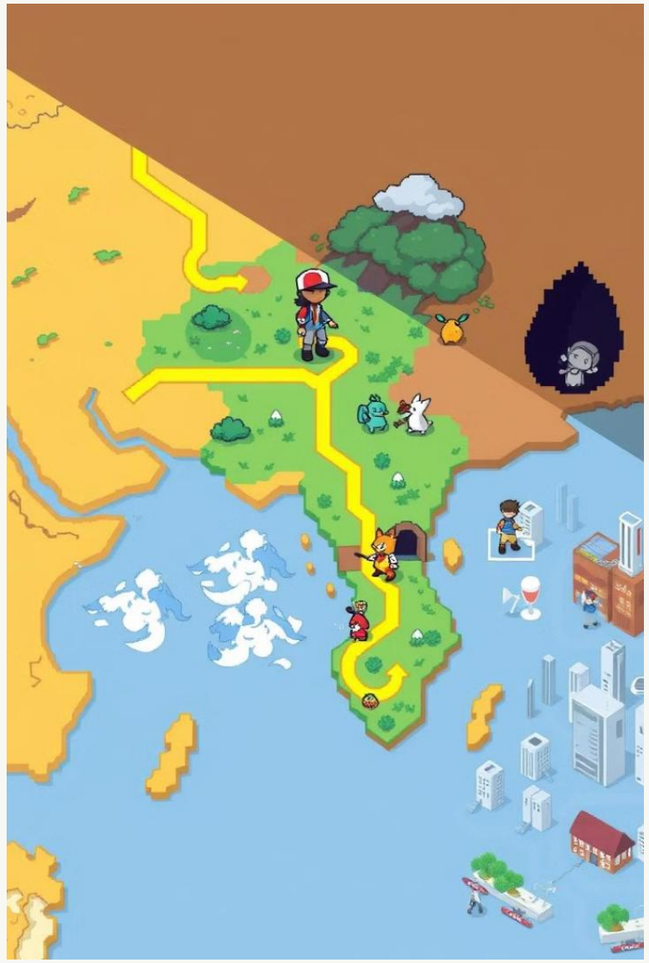
트레이너 또는 로켓단으로 시작하는 구성을 기획했습니다. 선택에 따라 전개가 달라지는 로그라이크 방식의 스토리를 구상했습니다.

구현한 기능

타일 맵 제작·편집 도구와 게임 기본 구조를 만들고, 전투 화면의 포켓몬 배치와 HP·경험치·레벨 표시를 구현했습니다.

전투 기능의 구현 범위

기술 사용과 아이템 사용 기능은 미완성 상태이며, 전투 화면과 상태 정보 표시를 구현했습니다.



타일 맵 에디터의 화면 구성

원본 타일 선택

왼쪽 화면에서 원본 이미지를 표시하고 사용할 타일을 선택합니다.

맵 편집 영역

오른쪽 화면에서 맵을 편집합니다. 검은 사각형으로 마우스가 선택한 타일 위치를 보여줍니다.

레이어 선택

화면 상단 버튼으로 수정할 레이어를 선택합니다. 1층은 기본 초록색 타일로, 나머지 층은 빈 상태로 시작합니다.



타일 선택과 레이어별 편집



원본 이미지 변경

숫자 키 1부터 9, T 키로 사용할 원본 이미지를 선택합니다.

선택한 타일 배치

원본 이미지에서 필요한 타일을 클릭한 뒤 맵 영역을 수정합니다.

레이어를 구분해 작업

상단의 레이어 버튼으로 편집 대상을 바꿉니다. 바닥과 그 위에 표시할 오브젝트를 구분해 제작합니다.

나무 배치를 위한 다중 타일 편집



전용 원본 이미지

T 키로 나무를 반복 배치하기 위한 이미지를 선택합니다.

4칸 단위 편집

나무 이미지는 타일 4칸을 함께 수정하는 방식으로 배치합니다.

반복 배치 방식

원본의 나무는 타일 크기의 2배 간격으로 떨어져 있습니다. 나무 사이를 선택하면 맵을 옆으로 이어서 수정할 수 있습니다.

타일 정보 저장과 불러오기

타일 데이터

각 타일은 원본 이미지 이름과 좌표 정보를 가집니다. 맵의 타일 정보를 텍스트 파일로 저장하고 다시 불러오도록 구성했습니다.

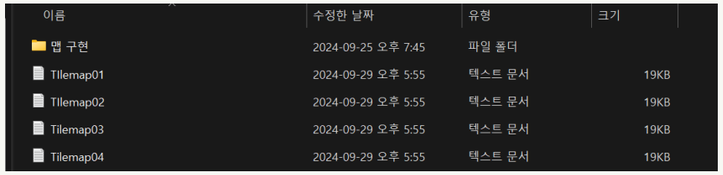
Q 키로 저장

현재 타일 맵의 정보를 .txt 파일로 저장합니다.

E 키로 불러오기

저장된 .txt 파일을 읽어 타일 맵을 불러옵니다.

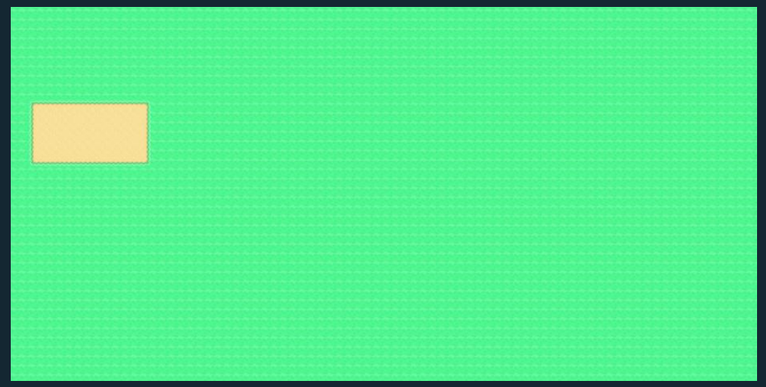




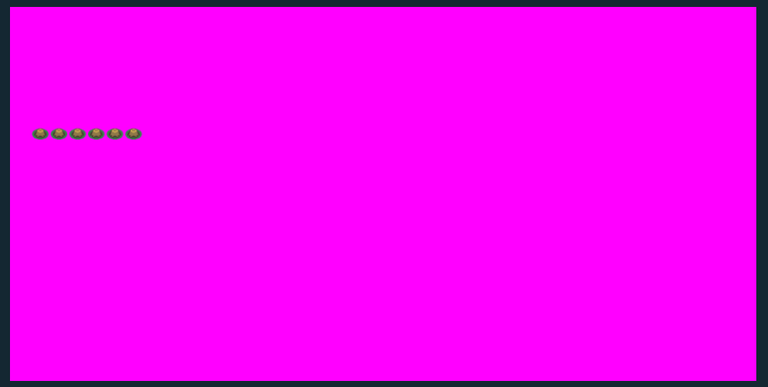
타일 맵과 저장된 텍스트 파일

레이어별 이미지 저장

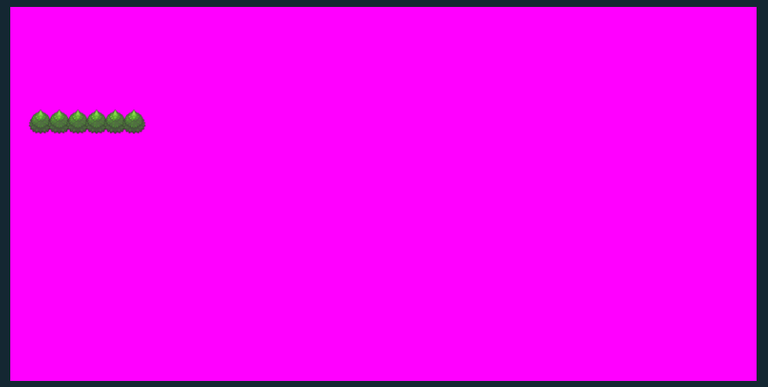
레이어를 바꿔가며 I 키를 누르면 해당 레이어를 이미지로 저장합니다.



바닥 레이어



오브젝트 레이어



상단 레이어

플레이 중 타일마다 리소스를 다시 불러와 그리는 대신, 제작한 맵을 이미지로 저장해 출력했습니다.

더블 버퍼링을 적용한 화면 출력

버퍼 준비

기존 hdc와 호환되는 DC와 비트맵을 만들고 연결합니다.

```
::GetClientRect(hwnd, &_rect);

hdcBack = ::CreateCompatibleDC(hdc);                                    // hdc와 호환되는 DC 생성
_bmpBack = ::CreateCompatibleBitmap(hdc, _rect.right, _rect.bottom);    // hdc와 호환되는 비트맵 생성
HBITMAP prev = (HBITMAP)::SelectObject(hdcBack, _bmpBack);              // dcBack과 비트맵 연결
::DeleteObject(prev);
```

한곳에 렌더링

게임의 Render 처리는 화면 대신 hdcBack에 그리도록 구성했습니다.

```
void Game::Render()
{
    GET_SINGLE(SceneManager)->Render(hdcBack);
```

화면 복사와 초기화

BitBlt로 hdcBack을 hdc에 복사하고, PatBlt로 다음 렌더링을 위한 버퍼를 검은색으로 변경합니다.

```
// Double Buffering
::BitBlt(hdc, 0, 0, _rect.right, _rect.bottom, hdcBack, 0, 0, SRCCOPY);     // 비트 블릿 : 고속 복사
::PatBlt(hdcBack, 0, 0, _rect.right, _rect.bottom, BLACKNESS);              // hdcBack을 검은색으로 만들어 줌
```

전방 선언과 게임 데이터 관리

헤더의 타입 선언

enum으로 개수 등의 상수를 정의하고, 구조체를 전방 선언해 포인터로 참조하도록 구성했습니다.

```
enum
{
    PokemonMaxCount = 251,
    SkillMaxCount = 251
};

struct PokemonInfo;
struct PokemonStat;

struct SkillInfo;
```

이름과 번호로 데이터 접근

map은 문자열을 키로 받아 필요한 데이터를 찾고, vector는 번호를 인덱스로 사용해 데이터에 접근합니다.

```
private:
    std::vector<PokemonInfo*>                _infos = {};
    std::vector<PokemonStat*>                _stats = {};
    std::vector<SkillInfo*>                  _skills = {};

    std::map<std::wstring, PokeType>         _types;
    std::map<std::wstring, SkillCategory>    _categories;

    std::vector<std::vector<float>>          _typeChart = {};
```

한글 문자열과 타입 데이터 연결

wstring으로 이름 관리

한글 입력을 처리하기 위해 wstring 배열로 이름을 구성했습니다.

map으로 타입 조회

한글 이름과 타입을 map으로 연결하고, 입력된 이름에서 대응하는 타입을 얻도록 구성했습니다.

```cpp
// 한글 이름으로 포켓몬 타입 변환
const int32 size = static_cast<int32>(PokeType::End);
std::wstring typeName[size] =
{
    L"",
    L"노말",
    L"불꽃",
    L"물",
    L"풀",
    L"전기",
    L"얼음",
    L"격투",
    L"독",
    L"땅",
    L"비행",
    L"에스퍼",
    L"벌레",
    L"바위",
    L"고스트",
    L"드래곤",
    L"악",
    L"강철",
    L"페어리",
};

// 한글 이름으로 기술 카테고리 변환
const int32 size2 = static_cast<int32>(SkillCategory::End);
std::wstring categoryName[size2] =
{
    L"물리",
    L"특수",
    L"변화"
};
```

```cpp
// map으로 포켓몬 타입을 변환하기 위한 세팅
for (int32 i = 0; i < size; i++)
{
    _types[typeName[i]] = static_cast<PokeType>(i);
}

// map으로 기술 카테고리를 변환하기 위한 세팅
for (int32 i = 0; i < size2; i++)
{
    _categories[categoryName[i]] = static_cast<SkillCategory>(i);
}

LoadPokemonInfo(L"..\\Resources\\Info\\PokemonInfo.csv");
LoadPokemonSkill(L"..\\Resources\\Info\\PokemonSkill.csv");
```

```cpp
// 타입 1
start = start + 9;
end = line.find(L',', start);
_infos[no]->type1 = _types[line.substr(start, end - start)];
```

한글 이름 정의와 타입 매핑 코드

Actor와 UI의 객체 관리

Actor

일반 포인터를 vector에 모아 사용하고, 종류별로 Layer를 구분해 배열로 관리했습니다.

```
private:
    std::vector<Actor*>                     _actors[LAYER_MAXCOUNT];
    std::vector<std::shared_ptr<UI>>        _uis;
```

UI

스마트 포인터인 shared_ptr를 vector에 모아 관리했습니다.

삭제 처리

Actor는 delete 후 nullptr로 바꿉니다. UI는 보관한 스마트 포인터를 nullptr로 변경합니다.

```
Scene::~Scene()
{
    for (const std::vector<Actor*>& actors : _actors)
        for (Actor* actor : actors)
            SAFE_DELETE(actor);

    _actors->clear();

    for (std::shared_ptr<UI>& ui : _uis)
        ui = nullptr;

    _uis.clear();
}
```

FSM을 활용한 GameObject 상태 관리

상태 정의

GameObject의 상태를 미리 정하고, 현재 상태 안에서 동작하도록 구성했습니다.

```
protected:
    Vec2Int          _cellPos = {};
    Vec2             _speed = {};
    Dir              _dir = DIR_Down;
    ObjectState      _state = ObjectState::Idle;
```

상태별 동작 분리

Idle, Move, Run, Battle 상태에 따라 Tick에서 대응하는 처리 함수를 호출합니다.

```
void GameObject::Tick()
{
    Super::Tick();

    switch (_state)
    {
    case ObjectState::Idle:
        TickIdle();
        break;
    case ObjectState::Move:
        TickMove();
        break;
    case ObjectState::Run:
        TickRun();
        break;
    case ObjectState::Battle:
        TickBattle();
        break;
    }
}
```

버튼 상태와 콜백 연결

버튼 상태

Default, Pressed, Clicked를 enum으로 정의했습니다.

```
enum ButtonState
{
    BS_Default,
    BS_Pressed,
    BS_Clicked,

    BS_MaxCount
};
```

상태별 이미지

각 버튼 상태에 대응하는 Sprite를 준비해 보관합니다.

```
protected:
    Sprite*         _currentSprite = nullptr;
    Sprite*         _sprites[BS_MaxCount] = {};
    ButtonState     _state = BS_Default;

    float           _sumTime = 0.f;

public:
    template<typename T>
    void            AddOnClickDelegate(T* owner, void(T::* func)())
    {
        _onClick = [owner, func]()
            {
                (owner->*func)();
            };
    }

    // 함수 포인터 + 함수 객체
    std::function<void(void)>   _onClick = nullptr;
};
```

클릭 동작 연결

AddOnClickDelegate로 실행할 함수를 연결하고, std::function으로 콜백을 보관합니다.

입력에 따른 버튼 상태 전환

마우스 입력 처리

버튼 위의 마우스 위치와 클릭 입력을 확인해 기본·눌림·클릭 상태를 전환합니다.

클릭 시 콜백 실행

클릭 상태에서 연결된 함수를 호출해 해당 버튼의 동작을 실행합니다.

Sprite 갱신

SetButtonState에서 현재 상태와 일치하는 Sprite로 변경합니다.

```cpp
void Button::Tick()
{
    Super::Tick();

    float deltaTime = GET_SINGLE(TimeManager)->GetDeltaTime();


    if (IsMouseInRect())
    {
        if (GET_SINGLE(InputManager)->GetButtonPress(KeyType::LeftMouse))
        {
            SetButtonState(BS_Pressed);
        }
        else
        {
            if (_state == BS_Pressed)
            {
                SetButtonState(BS_Clicked);
                if (_onClick)
                    _onClick();
            }
        }
    }
}
```

```cpp
void Button::SetButtonState(ButtonState state)
{
    _state = state;

    if (_sprites[state])
        SetCurrentSprite(_sprites[state]);
}
```

Scene 전환 중 UI 삭제 오류 해결

발생한 문제

버튼 콜백에서 ChangeScene을 바로 호출하면 버튼 UI가 모두 삭제되어 Scene::Update 실행 중 오류가 발생했습니다.

변경한 처리

버튼에서는 전환할 SceneType을 전달하도록 수정했습니다. Scene을 즉시 바꾸는 호출 대신 전환할 종류를 지정했습니다.

```
void EditPanel::OnClickTilemapButton()
{
    Scene* scene = GET_SINGLE(SceneManager)->GetCurrentScene();
    scene->SetSceneType(SceneType::TileMapScene);
}

void EditPanel::OnClickBattleButton()
{
    Scene* scene = GET_SINGLE(SceneManager)->GetCurrentScene();
    scene->SetSceneType(SceneType::BattleScene);
}

void EditPanel::OnClickDevSceneButton()
{
    Scene* scene = GET_SINGLE(SceneManager)->GetCurrentScene();
    scene->SetSceneType(SceneType::DevScene);
}
```

```
{
    std::shared_ptr<Button> ui = std::make_shared<Button>();
    ui->SetSprite(GET_SINGLE(ResourceManager)->GetSprite(L"DevSceneButton_Off"), BS_Default);
    ui->SetSprite(GET_SINGLE(ResourceManager)->GetSprite(L"DevSceneButton_On"), BS_Pressed);
    ui->SetSprite(GET_SINGLE(ResourceManager)->GetSprite(L"DevSceneButton_Off"), BS_Clicked);
    ui->SetPos({ _pos.x, _pos.y });

    ui->AddOnClickDelegate(this, &EditPanel::OnClickDevSceneButton);
    AddChild(ui);
}
```

전투 화면과 비율 기반 UI 배치

전투 화면 구성

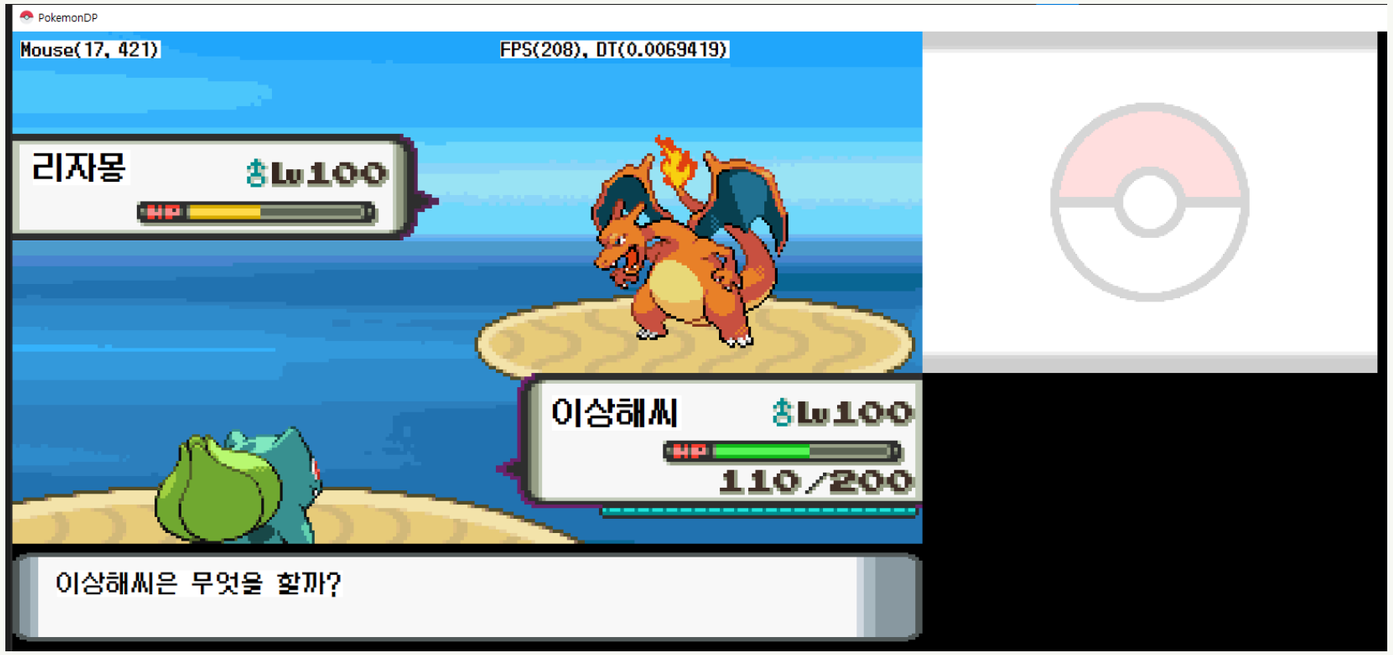
배경, 바닥, 포켓몬, UI와 Text Box를 화면 비율에 맞춰 배치했습니다.

상태 정보 표시

이름, HP, 레벨과 경험치처럼 변하는 값을 전투 화면에 표시했습니다. 숫자는 별도의 Sprite로 출력했습니다.

화면 비율 변경 대응

UI마다 절대 위치를 사용하는 대신 비율로 위치를 구성해 화면 크기에 따른 배치 변화를 해결했습니다.



HP 비율에 따른 표시 색상 변경

```
// hp 크기에 따른 hp 바 선택 및 기준 hp 변경

if (hp > maxHp / 2)
{
    _standardHp = maxHp;
    _currentHpBar = _hpBar[2];
}
else if (hp > maxHp / 4)
{
    _standardHp = maxHp / 2;
    _currentHpBar = _hpBar[1];
}
else
{
    _standardHp = maxHp / 4;
    _currentHpBar = _hpBar[0];
}
```

색상 기준

현재 HP와 최대 HP를 비교해 녹색, 주황색, 빨간색 중 표시할 Sprite를 선택합니다.

구간별 처리

코드에서는 최대 HP의 1/2과 1/4을 기준으로 구간을 구분합니다.

기준 HP 저장

선택한 HP Bar의 렌더링에 사용할 기준 HP 값을 함께 저장합니다.

HP Bar와 숫자 Sprite 렌더링

```cpp
// HP Bar
int32 hp = _pokemon->GetHp();
Vec2Int size = _currentHpBar->GetSize();
::AlphaBlend(hdc,
    _pos.x + _size.x * 31 / 60,
    _pos.y + _size.y * 19 / 41,
    size.x * _ratio * hp / _standardHp,
    size.y * _ratio,
    _currentHpBar->GetDC(),
    _currentHpBar->GetPos().x,
    _currentHpBar->GetPos().y,
    size.x * hp / _standardHp,
    size.y,
    bf);

// HP 표시
std::wstring hp_string = std::to_wstring(hp);
size = _numbers[0]->GetSize();
for (int32 i = 0; i < hp_string.size(); i++)
{
    ::AlphaBlend(hdc,
        _pos.x + _size.x * 47 / 60 + size.x * _ratio * (-i - 2),
        _pos.y + _size.y * 6 / 9,
        size.x * _ratio,
        size.y * _ratio,
        _numbers[hp_string[hp_string.size() - 1 - i] - L'0']->GetDC(),
        _numbers[hp_string[hp_string.size() - 1 - i] - L'0']->GetPos().x,
        _numbers[hp_string[hp_string.size() - 1 - i] - L'0']->GetPos().y,
        size.x,
        size.y,
        bf);
}
```

HP Bar 길이 계산

현재 HP와 기준 HP 값의 비율만큼 Bar를 렌더링합니다.

숫자 Sprite 선택

현재 HP를 wstring으로 변환한 뒤 미리 저장한 숫자 Sprite를 선택합니다.

자릿수별 배치

1의 자리부터 자릿수에 맞춰 출력하고, 현재 HP는 슬래시 왼쪽에 배치합니다.

윈도우 크기와 이미지 출력 문제 해결

윈도우가 모니터보다 커지는 문제

프로그램 시작 시 모니터 크기를 받아와 윈도우 크기를 설정하도록 수정했습니다.

PNG 파일을 출력하지 못하는 문제

GDI+ 라이브러리를 사용해 PNG 파일을 비트맵으로 변환해 출력했습니다.

화면 환경과 리소스 형식에 맞춰 초기화와 렌더링 방식을 조정했습니다.

타일 맵의 메모리와 출력 문제 해결

타일마다 객체를 생성한 메모리 부담

원본 이미지를 타일 크기로 잘라 객체를 만드는 대신, 이미지별로 객체를 만들고 필요한 영역만 잘라 출력했습니다.

플레이 중 타일별 이미지 로딩

제작한 타일 맵을 이미지로 저장하고, 플레이 중에는 저장된 맵 이미지를 출력하도록 변경했습니다.

바닥과 오브젝트의 그리기 순서

건물과 풀이 플레이어 위에 잘못 그려지는 문제는 타일 맵에 레이어를 추가하고, 레이어별로 제작하도록 바꿔 해결했습니다.

프로젝트 구현 내용과 리소스 출처

구현한 내용

타일 맵을 제작·저장·불러오는 에디터를 만들었습니다. 더블 버퍼링, 데이터와 객체 관리, FSM과 버튼 콜백을 적용하고 전투 화면의 상태 정보를 표시했습니다.

리소스

포켓몬스터 스프라이트: The Spriters Resource
게임 아트 제작·편집 도구: Photoshop

프로젝트 기록 보기

기획 윤상혁 개발 박우영